Experiment No. 2

Aim :

Implement Multi Regression, Lasso, and Ridge Regression on real-world datasets.

Theory :

1. Dataset Source

Dataset Name: **California Housing Dataset**
Source Platform: Kaggle

Dataset Link :
https://www.kaggle.com/code/hassaan2580/california-housing-dataset/input

2. Dataset Description

The California Housing dataset consists of various demographic and geographic features used to predict the median house value in a given district.

Features Included :

- longitude – Geographical coordinate
- latitude – Geographical coordinate
- housing_median_age – Median age of houses
- total_rooms – Total number of rooms
- total_bedrooms – Number of bedrooms
- population – Population in the area
- households – Number of households
- median_income – Median income of residents
- ocean_proximity – Categorical feature representing location

Target Variable :

- median_house_value – The median price of houses (Prediction target)

Dataset Characteristics :

- Contains numerical and categorical features
- Requires preprocessing such as handling missing values and encoding categorical data
- Suitable for regression analysis and regularization techniques

3. Mathematical Formulation of the Algorithm

Multiple Linear Regression

Multiple Linear Regression models the relationship between independent variables and a dependent variable using a linear equation :

$$y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i + \epsilon$$

Ridge Regression (L2 Regularization)

Ridge adds a penalty term to reduce overfitting:

$$\min ||y - X\beta||^2 + \alpha||\beta||^2$$

Lasso Regression (L1 Regularization)

Lasso introduces L1 penalty:

$$\min ||y - X\beta||^2 + \alpha|\beta|$$

4. Algorithm Limitations

Multiple Linear Regression :

- Sensitive to multicollinearity
- Can overfit when features are large
- Assumes linear relationship between variables

Ridge Regression :

- Does not perform feature selection
- Requires feature scaling

Lasso Regression :

- May remove useful features if penalty is high
- Performance depends heavily on alpha parameter

5. Methodology / Workflow

- Downloaded the California Housing dataset from Kaggle.
- Uploaded the CSV file into Google Colab environment.
- Loaded dataset using pandas library.
- Performed data preprocessing:
- Removed missing values
- Converted categorical column using one-hot encoding
    - Split data into training and testing sets.
    - Applied feature scaling using StandardScaler.
    - Trained three models:
- Multiple Linear Regression

- Ridge Regression
- Lasso Regression
    - Predicted housing prices using test data.
    - Evaluated model performance using R^2 Score and Mean Squared Error.
    - Applied GridSearchCV for hyperparameter tuning.

6. Performance Analysis

The performance of all three models was evaluated using R^2 Score and Mean Squared Error.

- Multiple Linear Regression provided a baseline performance.
- Ridge Regression reduced overfitting by penalizing large coefficients.
- Lasso Regression reduced model complexity and removed less important features.

Observation:

- Ridge Regression showed more stable performance compared to Linear Regression.
- Lasso Regression produced a simpler model by shrinking coefficients.
- Regularization improved generalization ability on unseen data.

7. Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV to identify the optimal alpha value for Ridge and Lasso regression.

Ridge Regression Tuning:

Different alpha values such as 0.01, 0.1, 1, 10, and 100 were tested using cross-validation to minimize prediction error.

Lasso Regression Tuning:

GridSearchCV evaluated multiple alpha values and selected the best parameter that achieved optimal model performance.

Impact of tuning:

- Improved R^2 Score
- Reduced prediction error
- Increased model stability

Output :

[1]

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
```

[4]

```python
df = pd.read_csv("/housing.csv")
df.head()
```

ude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value	ocean_proximity
.88	41.0	880.0	129.0	322.0	126.0	8.3252	452600.0	NEAR BAY
.86	21.0	7099.0	1106.0	2401.0	1138.0	8.3014	358500.0	NEAR BAY
.85	52.0	1467.0	190.0	496.0	177.0	7.2574	352100.0	NEAR BAY
.85	52.0	1274.0	235.0	558.0	219.0	5.6431	341300.0	NEAR BAY
.85	52.0	1627.0	280.0	565.0	259.0	3.8462	342200.0	NEAR BAY

Next steps: Generate code with df New interactive sheet

[6]

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

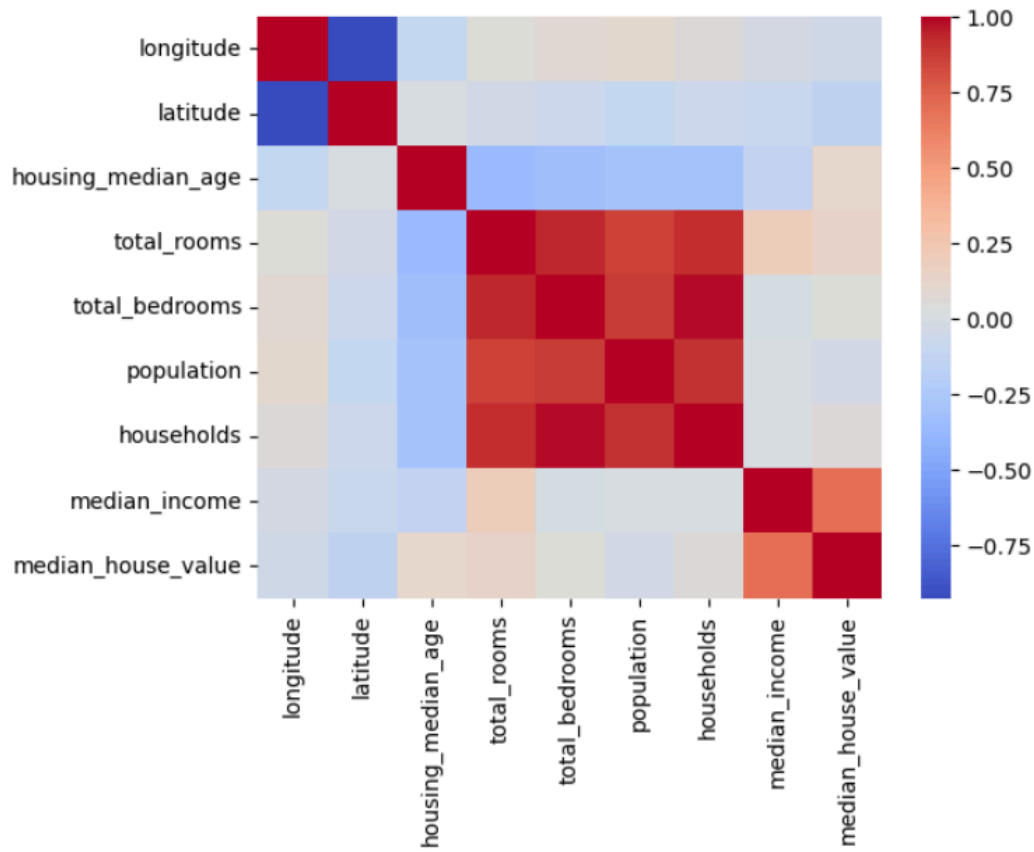
[7]
0s

```
df.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_
count	20640.000000	20640.000000	20640.000000	20640.000000	20433.000000	20640.000000	20640.000000	20640.000000	2
mean	-119.569704	35.631861	28.639486	2635.763081	537.870553	1425.476744	499.539680	3.870671	20
std	2.003532	2.135952	12.585558	2181.615252	421.385070	1132.462122	382.329753	1.899822	11
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900	1
25%	-121.800000	33.930000	18.000000	1447.750000	296.000000	787.000000	280.000000	2.563400	11
50%	-118.490000	34.260000	29.000000	2127.000000	435.000000	1166.000000	409.000000	3.534800	17
75%	-118.010000	37.710000	37.000000	3148.000000	647.000000	1725.000000	605.000000	4.743250	26
max	-114.310000	41.950000	52.000000	39320.000000	6445.000000	35682.000000	6082.000000	15.000100	50

[20]
0s

```
sns.heatmap(df.select_dtypes(include=['number']).corr(), cmap="coolwarm")
plt.show()
```



[9]

```python
df = df.dropna()
```

[10]

```python
X = df.drop("median_house_value", axis=1)
y = df["median_house_value"]
```

[11]

```python
X = pd.get_dummies(X, drop_first=True)
```

[12]

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[13]

```python
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[14]

```python
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)

y_pred_linear = linear_model.predict(X_test)

print("Linear R2:", r2_score(y_test, y_pred_linear))
```

```
Linear R2: 0.6488402154431994
```

[15]

```python
ridge_model = Ridge(alpha=1.0)
ridge_model.fit(X_train, y_train)

y_pred_ridge = ridge_model.predict(X_test)

print("Ridge R2:", r2_score(y_test, y_pred_ridge))
```

```
Ridge R2: 0.6488399119509963
```

[16]

```python
lasso_model = Lasso(alpha=0.1)
lasso_model.fit(X_train, y_train)

y_pred_lasso = lasso_model.predict(X_test)

print("Lasso R2:", r2_score(y_test, y_pred_lasso))
```

```
Lasso R2: 0.6488401950304886
```

[17]

```python
ridge_params = {"alpha":[0.01,0.1,1,10,100]}
ridge_grid = GridSearchCV(Ridge(), ridge_params, cv=5)
ridge_grid.fit(X_train, y_train)
print(ridge_grid.best_params_)
```

```
{'alpha': 10}
```

[18]
3s

```
lasso_params = {"alpha":[0.001,0.01,0.1,1,10]}
lasso_grid = GridSearchCV(Lasso(), lasso_params, cv=5)
lasso_grid.fit(X_train, y_train)
print(lasso_grid.best_params_)
```

```
{'alpha': 10}
```

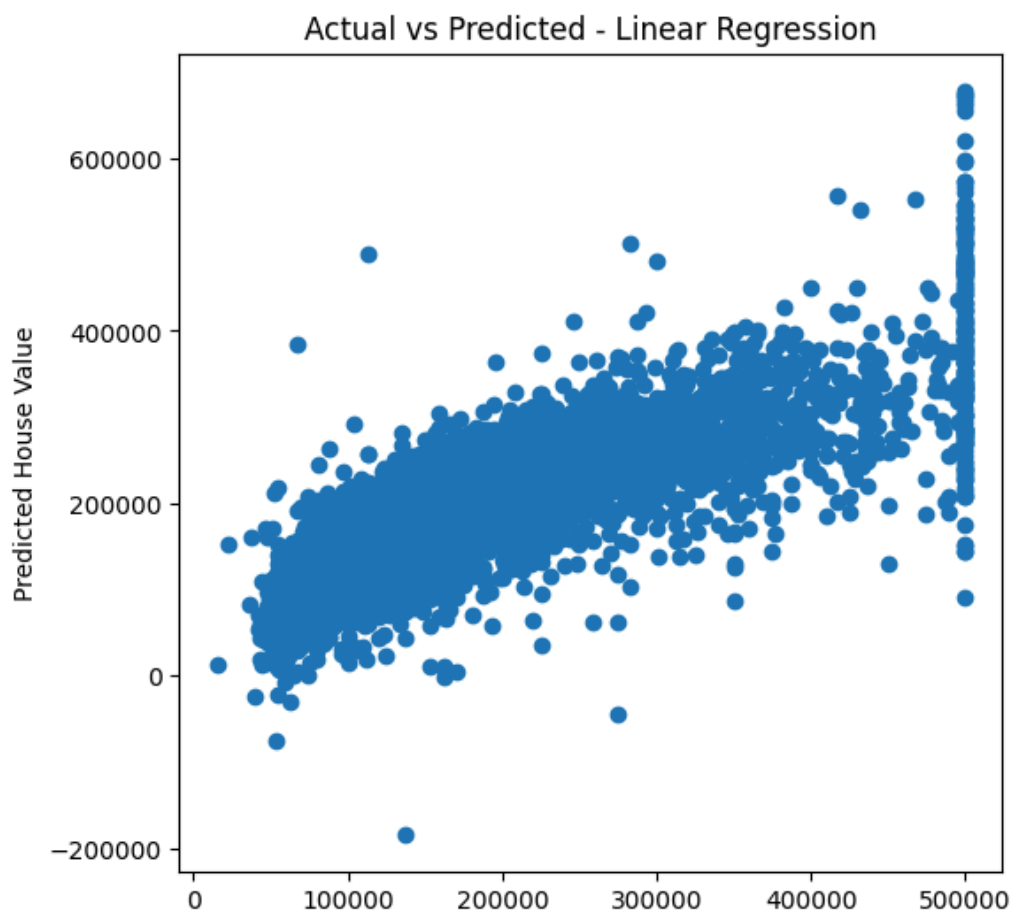
[19]
0s

```
results = pd.DataFrame({
    "Model":["Linear","Ridge","Lasso"],
    "R2":[
        r2_score(y_test,y_pred_linear),
        r2_score(y_test,y_pred_ridge),
        r2_score(y_test,y_pred_lasso)
    ]
})

results
```

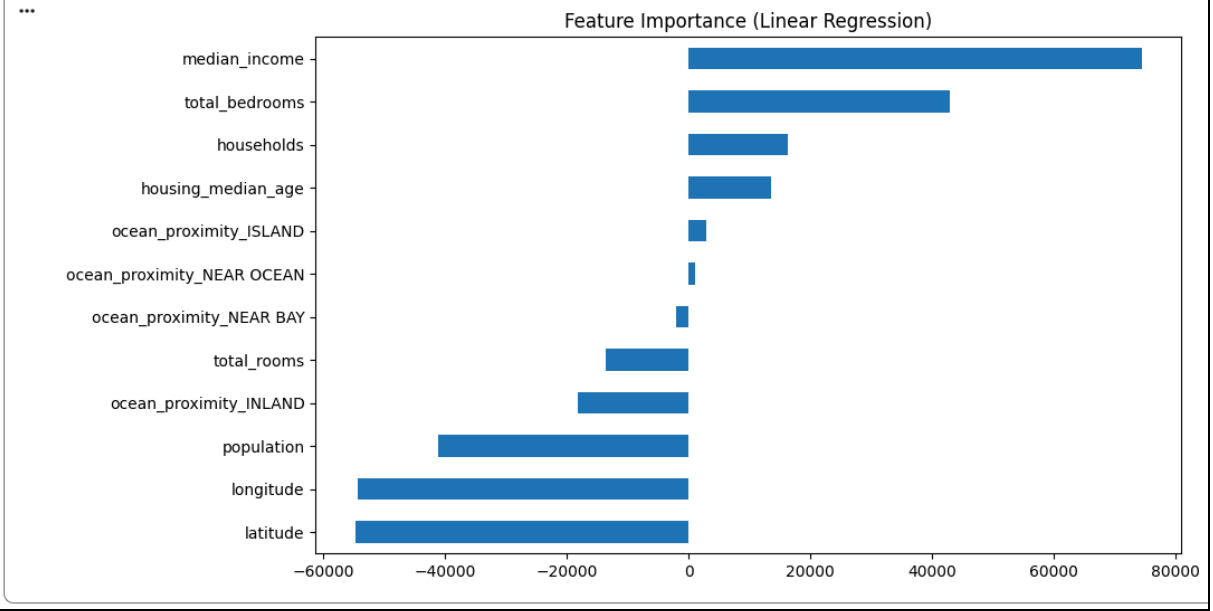
	Model	R2
0	Linear	0.64884
1	Ridge	0.64884
2	Lasso	0.64884

[21]
✓ 0s

```python
plt.figure(figsize=(6,6))

plt.scatter(y_test, y_pred_linear)
plt.xlabel("Actual House Value")
plt.ylabel("Predicted House Value")
plt.title("Actual vs Predicted - Linear Regression")

plt.show()
```



```
[24]
plt.figure(figsize=(10,6))

coeff = pd.Series(linear_model.coef_, index=X.columns)
coeff.sort_values().plot(kind="barh")

plt.title("Feature Importance (Linear Regression)")
plt.show()
```



Conclusion :

This experiment successfully implemented Multiple Linear Regression, Ridge Regression, and Lasso Regression on a real-world housing dataset. Data preprocessing and feature scaling improved the model's learning and prediction accuracy. Regularization techniques such as Ridge and Lasso helped reduce overfitting and improved generalization performance. Overall, the comparison showed that regularized models provide more stable and reliable predictions than basic linear regression.